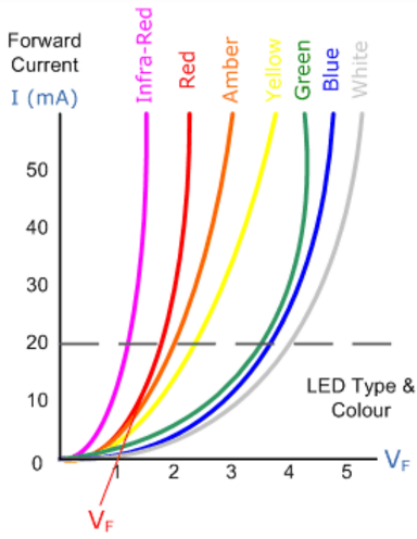
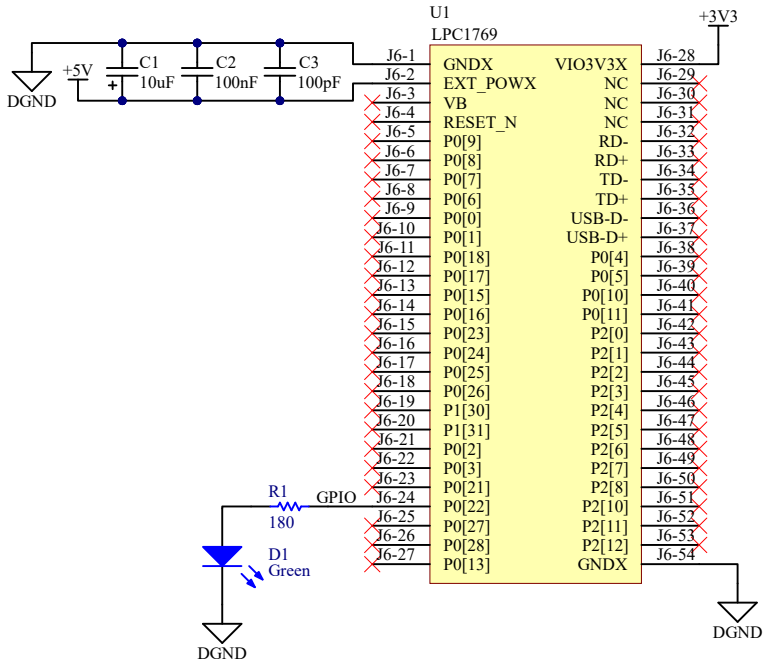


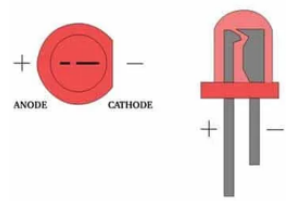
```

1  /*=====
2  * @file    Ejercicio_2_1.c
3  *
4  * @brief    Controla un LED conectado al pin P0.22, alternando periódicamente
5  *           entre los estados encendido y apagado.
6  *
7  * @details  Implementar retardo bloqueante.
8  *
9  * @author   Ing. E. Migliore
10 *
11 * @version  3.0
12  */

```



$$R_3 = \frac{V_{RC0} - V_D}{I_D} = \frac{3,3V - 2,24V}{6mA} \cong 180\Omega$$



```

1  /*=====
2  * DIRECTIVAS DE INCLUSIÓN
3  *=====*/
4  #include "LPC17xx.h"
5
6  /*=====
7  * DEFINICIÓN DE CONSTANTES
8  *=====*/
9  #define LED      (1<<22)
10
11 /*=====
12 * DEFINICIÓN DE VARIABLES GLOBALES
13 *=====*/
14
15 /*=====
16 * PROTOTIPOS DE FUNCIONES
17 *=====*/
18 void cfgGPIO(void);
19 void blinkLed(void);
20 void delay(void);
21
22 /*=====
23 * FUNCIÓN PRINCIPAL
24 *=====*/
25 int main(void)
26 {
27     /*-----
28     * INICIALIZACIÓN DEL MCU
29     *-----*/
30     cfgGPIO();
31
32     /*-----
33     * INICIO PROGRAMA PRINCIPAL
34     *-----*/
35     while(1)
36     {
37         blinkLed();
38     }
39
40     return 0;
41 }
42
43 /*=====
44 * DEFINICIONES DE FUNCIONES
45 *=====*/
46
47 /******
48 * @brief    Configura el pin P0.22 como salida digital.
49 *
50 * @details  Selecciona la función GPIO para P0.22, configura el pin como
51 *           salida digital y habilita su acceso mediante el registro FIOMASK.
52 *          *****/
53 void cfgGPIO(void)
54 {
55     LPC_PINCON->PINSEL1  &= ~(0b11<<12); // P0.22 GPIO Function
56     LPC_GPIO0->FIODIR    |= (1<<22);      // P0.22 Digital Output
57     //LPC_GPIO0->FIODIRL |= (1<<6);
58     LPC_GPIO0->FIOMASK   &= ~(1<<22);
59
60     return;
61 }
62
63
64
65
66
67
68
69

```

```

70  /*****
71  * @brief      Ejecuta un ciclo de intermitencia del LED conectado a P0.22.
72  *
73  * @details    Enciende el LED, genera un retardo y posteriormente lo apaga,
74  *             generando nuevamente un retardo antes de finalizar el ciclo.
75  *****/
76  void blinkLed(void)
77  {
78      LPC_GPIO0->FIOSET |= LED;    // Led ON
79      delay();
80      LPC_GPIO0->FIOCLR |= LED;    // Led OFF
81      delay();
82
83      return;
84  }
85
86  /*****
87  * @brief      Genera un retardo bloqueante mediante instrucciones NOP.
88  *
89  * @details    El retardo se implementa mediante un lazo de espera que ejecuta
90  *             una instrucción NOP en cada iteración. La instrucción NOP no
91  *             modifica el estado del procesador y consume un ciclo de CPU.
92  *             La duración del retardo depende de la frecuencia de CPU y de
93  *             la cantidad de iteraciones ejecutadas.
94  *****/
95  void delay(void)
96  {
97      for(volatile uint32_t i = 0; i < 3500000; i++)
98      {
99          __NOP();    // NOP: 1 ciclo de CPU
100      }
101
102      return;
103  }
104
105

```